

1. **State and re-renders:** State, props and derived values · `useState` and what a re-render is · The rules of hooks (and why)
2. **One render's snapshot:** State is a snapshot of one render · Updater functions: new state from the previous state · Batching: many updates, one render
3. **What to store and how to update it:** Don't store what you can compute · Updating objects in state: replace, never mutate · Updating arrays: spread, map and filter
4. **Effects and dependencies:** Effects: synchronising with something outside React · The dependency array: `none`, `[]` and `[x]` · Stale closures in effects and timers · You might not need an effect
5. **Cleanup, refs and StrictMode:** Cleanup: before the next effect and on unmount · `useRef`: a box that survives renders without causing one · StrictMode: why effects run twice in development
6. **Effects in practice:** Fetching data in an effect · Custom hooks: reusing stateful logic · A custom hook for persistence: `useLocalStorage`

State and re-renders

State, props and derived values

An interface shows three kinds of data: **props** (given by the parent, read-only), **state** (remembered by the component and changed over time) and **derived values** (computed from the other two during render).

1. **Does it come from the parent?** A prop: read it, never change it.
2. **Can it be computed from props or other state?** Derived: a plain `const` in the component body.
3. **Does it change over time, with nobody else giving it?** Only then it is state, declared with `useState` (see [useState and what a re-render is](#)).

Which kind of data is it?

Value	Kind	Why
The list of tasks passed by the parent	Prop	Someone else owns it
The text typed in the search box	State	Changes over time; nobody else knows it
The tasks that match the search	Derived	Computed from the list and the text
"3 of 5 done"	Derived	Counted from the list
Whether "Show done" is ticked	State	The user changes it

```
import { useState } from 'react';

const TASKS = [
  { id: 1, title: 'Write the API skeleton', done: true },
  { id: 2, title: 'Add full CRUD', done: false },
  { id: 3, title: 'Document the endpoints', done: true },
];

function TaskSummary({ tasks }) {
  // prop
  const [showDone, setShowDone] = useState(true); // state
  const visible = showDone ? tasks : tasks.filter((t) => !t.done); // derived
  const doneCount = tasks.filter((t) => t.done).length; // derived

  return (
    <section>
      <label>
        <input type="checkbox" checked={showDone}
          onChange={(e) => setShowDone(e.target.checked)} /> Show done tasks
      </label>
      <p>{doneCount} of {tasks.length} done</p>
      <ul>
        {visible.map((t) => <li key={t.id}>{t.done ? '✓ ' : ''}{t.title}</li>)}
      </ul>
    </section>
  );
}
```

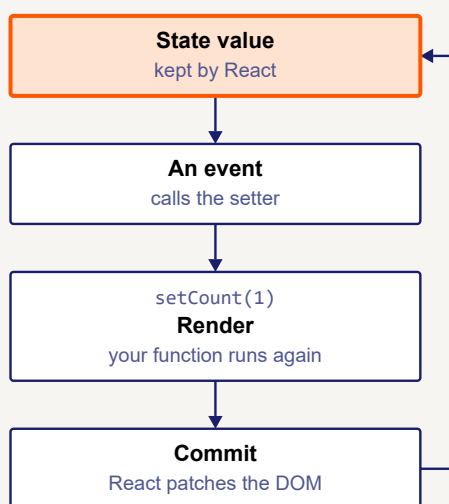
```
function App() {
  return <TaskSummary tasks={TASKS} />;
}
```

Example. The Try it box receives tasks as a prop, keeps one piece of state (showDone) and derives two values (visible and doneCount). Untick the checkbox: only the state changes, and React recomputes both derived values on the next render.

Common mistake. Copying a prop into state: `const [tasks, setTasks] = useState(props.tasks)`. `useState` reads its argument **only on the first render**, so when the parent later passes a new list, this component keeps showing the old one. Read the prop directly; copy it into state only for an editable draft that should ignore later changes.

useState and what a re-render is

`const [value, setValue] = useState(initial)` gives a component one piece of memory; calling `setValue(next)` stores the new value and asks React to **re-render**.



Every change goes round the same loop: setter, render, commit.

Render React calls your component again, top to bottom, with the new value, to get a fresh description of the screen. Its children render too.

Commit React compares that description with the previous one and changes only the parts of the DOM that differ (see [Elements, rendering and the DOM](#)).

Same value A setter called with the value it already has (by `Object.is`) skips the render.

- **Naming:** array destructuring, `[thing, setThing]: const [count, setCount] = useState(0)`.
- **The initial value is used once**, on the first render. If computing it is expensive (reading storage, for example), pass a function: `useState(() => load())`, a **lazy initialiser** that React calls only that first time.
- **Per instance:** two `<Counter />` elements count separately.

```
import { useState } from 'react';

function App() {
  const [count, setCount] = useState(0);
  let clicks = 0; // a plain local variable
  console.log(`App rendered: count = ${count}, clicks = ${clicks}`);

  return (
    <>
      <button onClick={() => { clicks++; console.log('clicks is now', clicks); }}>
        Plain variable: {clicks}
      </button>
    </>
  );
}
```

```

    <button onClick={() => setCount(count + 1)}>
      State: {count}
    </button>
  </>
);
}

```

Example. Click both buttons in the Try it box and watch its console. The `let clicks` variable is increased, but nothing re-renders and the next render starts it from 0 again. The state button re-renders, and the log shows the component running again with the new value.

Common mistake. Assigning to the state variable: `count = count + 1`. With `const` it throws a `TypeError`; with `let` it runs, but nothing tells React, so the screen never changes and the next render brings the old value back. Always go through the setter: the variable is a read-only copy for this render.

The rules of hooks (and why)

Call hooks (`useState`, `useEffect`, **any** `use...` **function**) **only at the top level of a component or a custom hook, never inside conditions, loops, nested functions or after an early return.**

- **Top level, same order:** no hook inside `if`, loops or nested functions.
- **Before any early return:** `if (!user) return <Login />`; above a `useState` makes the number of hooks depend on `user`.
- **Only from components or custom hooks** (functions named `use...`), never from ordinary functions or event handlers.
- **Conditions go inside the hook:** `useEffect(() => { if (!enabled) return; ... }, [enabled])`, or render a child conditionally, so its hooks live in the child.

```

// Wrong: the hook only runs sometimes
function TaskDetail({ task }) {
  if (!task) return <p>Select a task</p>;
  const [editing, setEditing] = useState(false); // slot depends on task
  ...
}

// Right: every hook first, then the early return
function TaskDetail({ task }) {
  const [editing, setEditing] = useState(false);
  if (!task) return <p>Select a task</p>;
  ...
}

```

```

import { useState } from 'react';

function App() {
  const [details, setDetails] = useState(false);

  if (details) {
    // Breaks the rules: this hook only exists on some renders
    const [note, setNote] = useState('a note');
    console.log(note);
  }

  return (
    <label>
      <input type="checkbox" checked={details} onChange={(e) => setDetails(e.target.checked)} />
      Show details (this breaks the component)
    </label>
  );
}

```

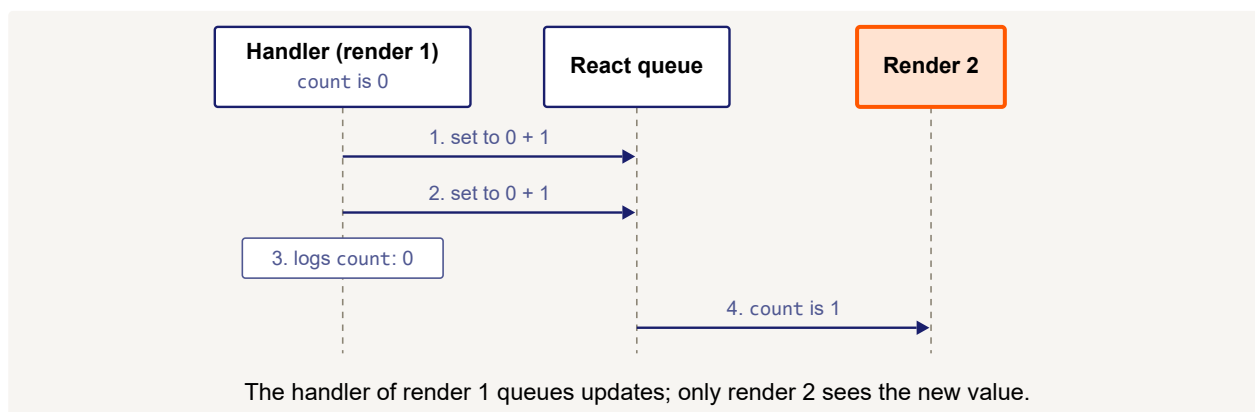
Example. The Try it box breaks the rule on purpose: an extra `useState` runs only when the checkbox is ticked. The first render works; tick the checkbox and React throws, because the second render calls one more hook than the first. Move the hook out of the `if` to fix it.

Common mistake. Calling a hook inside an event handler: `onClick={() => { const [x, setX] = useState(0); ... }}`. Handlers run outside rendering, so React cannot attach the state to the component and throws "Invalid hook call". Declare the state at the top level and use its setter in the handler.

One render's snapshot

State is a snapshot of one render

Inside one render a state variable is a **constant**: calling the setter does not change it, it queues a new value for the **next** render.



- **Two setters, one value:** `setCount(count + 1); setCount(count + 1);` reads `count` twice from the same render (0) and twice queues "set it to 1".
- **The log after a setter** still prints the old value.
- **A timer started in a click** (`setTimeout(() => alert(count), 3000)`) shows that render's value, however many times you click meanwhile.

```
import { useState } from 'react';

function App() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    setCount(count + 1);
    setCount(count + 1);
    console.log('right after three setters, count is', count);
    setTimeout(() => console.log('3 s later this click still sees', count), 3000);
  }

  return <button onClick={handleClick}>+3? Count: {count}</button>;
}
```

Example. In the Try it box, click **+3?**: it calls `setCount(count + 1)` three times and logs `count` afterwards. The console says 0, and the screen shows 1. Then open the render-cycle visualiser in this card, choose "Two `setCount(count + 1)` calls" and step through the queue.

Common mistake. Reading state right after setting it to decide something: `setQuery(text); fetchResults(query);` sends the **old** query. Use the value you just computed (`fetchResults(text)`), or let an effect react to the new query after the render.

Updater functions: new state from the previous state

Pass the setter a function instead of a value, `setCount((c) => c + 1)`, and React calls it with the **latest** queued value; use it whenever the next state depends on the previous one.

- **Where it matters most:** code that runs later than its render: an interval started by an effect, a promise callback, a handler that queues several updates. The captured variable may be stale; the updater's argument never is.
- **Naming:** the parameter is up to you; common choices are the first letter (c for count) or prev.
- **Pure:** an updater only computes and returns the next value. No fetch, no changes to other variables: in development, StrictMode calls updaters twice to expose them.

```
setCount(count + 1);           // "make it 1" (if count is 0 in this render)
setCount((c) => c + 1);        // "add one to whatever it is by then"

// Mixing: value then updater
setCount(10);                 // queue: replace with 10
setCount((c) => c * 2);        // queue: 10 → 20   (next render: 20)
```

```
import { useState } from 'react';

function App() {
  const [count, setCount] = useState(0);

  function addThree() {
    setCount((c) => c + 1);
    setCount((c) => c + 1);
    setCount((c) => c + 1);
  }

  function resetThenOne() {
    setCount(0);           // a value: replace
    setCount((c) => c + 1); // an updater: receives 0
  }

  return (
    <>
      <p>Count: {count}</p>
      <button onClick={addThree}>+3</button>
      <button onClick={resetThenOne}>Reset then +1</button>
    </>
  );
}
```

Example. The Try it box has the same three-call button, this time with updaters, so a click adds 3. The **Reset then +1** button queues a value and then an updater: the queue goes "replace with 0" → "0 + 1", and the screen shows 1.

Common mistake. Using an updater everywhere "to be safe" while still reading the stale variable inside it: `setTotal((t) => t + price * count)` is fine only if price and count are current. The updater protects the variable it receives, not every other variable it closes over.

Batching: many updates, one render

React waits until the code that queued updates has finished (an event handler, a timer callback, a promise callback) and then renders **once** for all of them: **batching**.



Setters only queue; React renders once, when the code that queued them returns.

- **Everywhere since React 18:** in event handlers, inside `setTimeout`, after an `await`, in promise callbacks. Older versions batched only event handlers, which is why old articles call it an exception.
- **The escape hatch** for the rare case where the DOM must be updated before the next line (to measure it or scroll to a new element) is `flushSync` from `react-dom`.

```
import { useState } from 'react';

function App() {
  const [count, setCount] = useState(0);
  const [status, setStatus] = useState('idle');
  console.log(`render: count = ${count}, status = ${status}`);

  function handleClick() {
    setCount((c) => c + 1);
    setStatus('saving');           // same batch as the line above
    setTimeout(() => {
      setCount((c) => c + 1);
      setStatus('saved');         // batched too, inside the timer
    }, 1000);
  }

  return <button onClick={handleClick}>{status}: {count}</button>;
}
```

Example. Click the button in the Try it box. The handler updates two variables and also waits for a timer before updating them again. The console shows one render per batch: two renders for the whole click, not four.

Common mistake. Expecting a render (and a log in the component body) per setter call, and splitting related state into many variables "so they update separately". They do not: every update of the same event lands in the same render. If two values always change together, consider one state object or one derived value.

What to store and how to update it

Don't store what you can compute

If a value can be calculated from props or existing state, compute it **during render** instead of keeping it in state: duplicated state has to be kept in sync by hand, and it eventually goes out of sync.

- **Typical derived values:** filtered or sorted lists, counts and totals, a full name from its parts, "is the form valid?", "is this the selected item?".
- **Store an id, not a copy:** keep `selectedId` and find the item during render. A copy of the object goes stale when the list is edited; the id does not.
- **Recomputing is cheap:** filtering a few hundred items takes microseconds. Only a measured, truly heavy calculation deserves `useMemo` (see [Memoising: useMemo, useCallback and memo](#)).
- **Never an effect for it:** an effect that copies a computed value into state renders first with the stale value, then again with the right one (see [You might not need an effect](#)).

```
// Duplicated: a second list that every update must remember to change
const [tasks, setTasks] = useState(START);
const [visible, setVisible] = useState(START); // goes stale on the first missed update

// Derived during render: always right
const visible = tasks.filter((t) => t.title.toLowerCase().includes(search.toLowerCase()));
```

```
import { useState } from 'react';

const START = [
  { id: 1, title: 'Write the API skeleton', done: true },
  { id: 2, title: 'Add full CRUD', done: false },
  { id: 3, title: 'Return the right status codes', done: false },
];
```

```
function App() {
  const [tasks, setTasks] = useState(START);
  const [search, setSearch] = useState('');

  const visible = tasks.filter((t) => t.title.toLowerCase().includes(search.toLowerCase()));
  const pending = tasks.filter((t) => !t.done).length;

  const toggle = (id) => setTasks(tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));

  return (
    <>
      <input placeholder="Search" value={search} onChange={(e) => setSearch(e.target.value)} />
      <p>{pending} pending · showing {visible.length}</p>
      <ul>
        {visible.map((t) => (
          <li key={t.id}>
            <label><input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} /> {t.title}</label>
          </li>
        ))}
      </ul>
    </>
  );
}
```

Example. The Try it box filters a task list as you type. There are exactly two pieces of state, tasks and search; visible and the counter are computed in the body. Toggle a task and the counter stays correct with no extra code.

Common mistake. Storing the selected object: `const [selected, setSelected] = useState(task)`. Rename the task in the list and the detail panel still shows the old title, because it holds a separate copy. Store `selectedId` and derive `const selected = tasks.find((t) => t.id === selectedId)`.

Updating objects in state: replace, never mutate

Treat objects in state as read-only: to change one field, create a new object with the spread syntax, `setTask({ ...task, done: true })`; **changing the existing object** (`task.done = true`) **does not re-render**.

`{ ...task, done: true }` A new object with every property of task; properties written after the spread win.

Nested objects Copy **every level** you change: `{ ...user, address: { ...user.address, city: 'Madrid' } }`. The spread is shallow, so the untouched nested objects are shared, which is fine as long as nobody mutates them.

One handler for every field A computed property name: `setForm({ ...form, [e.target.name]: e.target.value })`, with `name="title"` on the input.

```
const [task, setTask] = useState({ id: 1, title: 'Add full CRUD', done: false });

// Wrong: same object, React sees no change
task.done = true;
setTask(task);

// Right: a new object
setTask({ ...task, done: true });

// Nested: copy each level you touch
setUser({ ...user, address: { ...user.address, city: 'Madrid' } });
```

```
import { useState } from 'react';

function App() {
  const [task, setTask] = useState({ title: 'Add full CRUD', done: false });

  function mutate() {
    task.done = !task.done; // edits the object of this render
```

```

    setTask(task);          // same reference: React skips the render
  }

  function replace() {
    setTask({ ...task, done: !task.done });
  }

  return (
    <>
      <input value={task.title} onChange={(e) => setTask({ ...task, title: e.target.value })} />
      <p>{task.title}: {task.done ? 'done' : 'pending'}</p>
      <button onClick={mutate}>Toggle (mutating)</button>
      <button onClick={replace}>Toggle (new object)</button>
    </>
  );
}

```

Example. The Try it box has a small edit form whose state is one object. **Toggle (mutating)** changes the object in place and calls the setter: nothing happens on screen. **Toggle (new object)** works. Type in the title after a mutating click and the hidden change suddenly appears, because typing causes a render.

Common mistake. Believing `const` protects the object. `const [task, setTask]` only stops you from reassigning the name; `task.done = true` is still allowed and is exactly the mutation React cannot see.

Updating arrays: spread, map and filter

Add with spread (`[...tasks, newTask]`), **remove with filter**, **change one item with map** and **a new object, and sort a copy; never push, splice, assign by index or sort the array in state.**

Array operations on state

Goal	Avoid (mutates)	Use (new array)
Add	<code>tasks.push(t)</code>	<code>[...tasks, t]</code>
Remove	<code>tasks.splice(i, 1)</code>	<code>tasks.filter((x) => x.id !== id)</code>
Change one	<code>tasks[i].done = true</code>	<code>tasks.map((x) => x.id === id ? { ...x, done: true } : x)</code>
Sort	<code>tasks.sort(cmp)</code>	<code>[...tasks].sort(cmp)</code> Or <code>tasks.toSorted(cmp)</code>
Insert at i	<code>tasks.splice(i, 0, t)</code>	<code>[...tasks.slice(0, i), t, ...tasks.slice(i)]</code>

- **Changing one item:** `map` gives a new array, and the changed item must also be a **new object**; untouched items are reused as they are.
- **Updates that run later** (after an `await`, in a timer) use the updater form, `setTasks((prev) => [...prev, created])`, so two quick additions both survive.

```

import { useState } from 'react';

let nextId = 3;

function App() {
  const [tasks, setTasks] = useState([
    { id: 1, title: 'Write the API skeleton', done: true },
    { id: 2, title: 'Add full CRUD', done: false },
  ]);

  const add = () => setTasks([...tasks, { id: nextId++, title: `Task ${nextId - 1}`, done: false }]);
  const addWithPush = () => {
    tasks.push({ id: nextId++, title: 'Pushed task', done: false });
    console.log('array length is now', tasks.length);
    setTasks(tasks); // same array: no render
  };
  const toggle = (id) => setTasks(tasks.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  const remove = (id) => setTasks(tasks.filter((t) => t.id !== id));

  return (
    <>
      <button onClick={add}>Add</button> <button onClick={addWithPush}>Add with push</button>
    </>
  );
}

```

```

    <ul>
      {tasks.map((t) => (
        <li key={t.id}>
          <label><input type="checkbox" checked={t.done} onChange={() => toggle(t.id)} /> {t.title}</label>
          <button onClick={() => remove(t.id)} aria-label={`Delete ${t.title}`}>X</button>
        </li>
      ))}
    </ul>
  </>
);
}

```

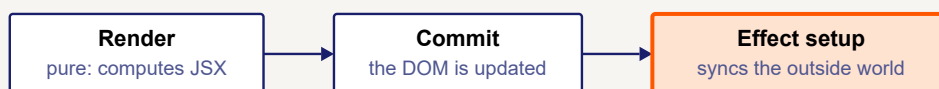
Example. The Try it box is a complete little task list with add, toggle and delete, all with new arrays. The **Add with push** button shows the broken version: the console proves the item went into the array, but the screen does not show it until some other update renders.

Common mistake. Sorting for display with `tasks.sort(...)` inside the JSX or the body. It reorders the state array in place during render, so the "original" order is lost and other components that share the array see it change. Sort a copy, or better, derive a sorted copy: `const sorted = tasks.toSorted(byTitle)`.

Effects and dependencies

Effects: synchronising with something outside React

An **effect** (`useEffect(setup, deps)`) is code that runs **after** React has updated the page, to keep something **outside React** in sync with the current props and state: a timer, a request, a browser API, a listener, a connection.



An effect runs after the page is up to date, never during render.

Setup Starts the synchronisation: connect, subscribe, start the timer.

Cleanup The function the setup returns. It stops the synchronisation: disconnect, unsubscribe, clear the timer (see [Cleanup](#)).

Dependencies The values the effect reads. When they change, React stops the old synchronisation and starts a new one (see [The dependency array](#)).

```

import { useEffect } from 'react';

useEffect(() => {
  // setup: runs after the commit
  const id = setInterval(tick, 1000);
  return () => clearInterval(id); // cleanup: undoes the setup
}, []); // dependencies: when to re-synchronise

```

```

import { useState, useEffect } from 'react';

function App() {
  const [tracking, setTracking] = useState(false);
  const [pos, setPos] = useState({ x: 0, y: 0 });

  useEffect(() => {
    if (!tracking) return; // nothing to synchronise
    console.log('subscribe to pointermove');
    const onMove = (e) => setPos({ x: e.clientX, y: e.clientY });
    window.addEventListener('pointermove', onMove);
    return () => {
      console.log('unsubscribe');
      window.removeEventListener('pointermove', onMove);
    };
  }, [tracking]);
}

```

```

    }, [tracking]);

    return (
      <div style={{ minHeight: 160 }}>
        <label>
          <input type="checkbox" checked={tracking} onChange={(e) => setTracking(e.target.checked)} />
          Track the pointer
        </label>
        <p>x = {pos.x}, y = {pos.y}</p>
      </div>
    );
  }

```

Example. The Try it box subscribes to the pointer position only while the checkbox is ticked. The effect depends on tracking: ticking starts a pointermove listener, unticking runs the cleanup, which removes it. Move the pointer over the preview and watch the console.

Common mistake. Calling fetch, setInterval or addEventListener directly in the component body. The body runs on **every** render: each keystroke adds another listener or timer, and a setState in the body triggers another render, which runs the body again: an infinite loop.

The dependency array: none, [] and [x]

The second argument of `useEffect` lists the values the effect reads, and decides when it runs: after every render, after the first render only, or whenever a listed value changed.

When does the effect run?

Second argument	Runs after	Typical use
(none)	Every render	Rare: syncing with something that depends on everything
[]	The first render only (and its cleanup on unmount)	A connection or listener that never changes
[roomId]	The first render, and when roomId changes	Fetch or subscribe for the current id

- **List every value the effect reads:** props, state and values derived from them. Leave one out and the effect keeps using an old value (see [Stale closures](#)). The ESLint rule `react-hooks/exhaustive-deps`, installed by Vite's React template, warns about missing ones: treat its warnings as bugs.
- **Need not be listed:** setters from `useState` and values defined outside the component never change.
- **Objects and functions made during render are new every time:** `[options]` with `const options = { search }` in the body behaves like no array. List the primitive `([search])` or build the object inside the effect.

```

import { useState, useEffect } from 'react';

function App() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState('');
  console.log(`render: count = ${count}, text = "${text}"`);

  useEffect(() => {
    console.log('A (no array): after every render');
  });
  useEffect(() => {
    console.log('B ([]): after the first render only');
  }, []);
  useEffect(() => {
    console.log(`C ([count]): count is ${count}`);
  }, [count]);

  return (
    <>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <input value={text} onChange={(e) => setText(e.target.value)} />
    </>
  );
}

```

```

    </>
  );
}

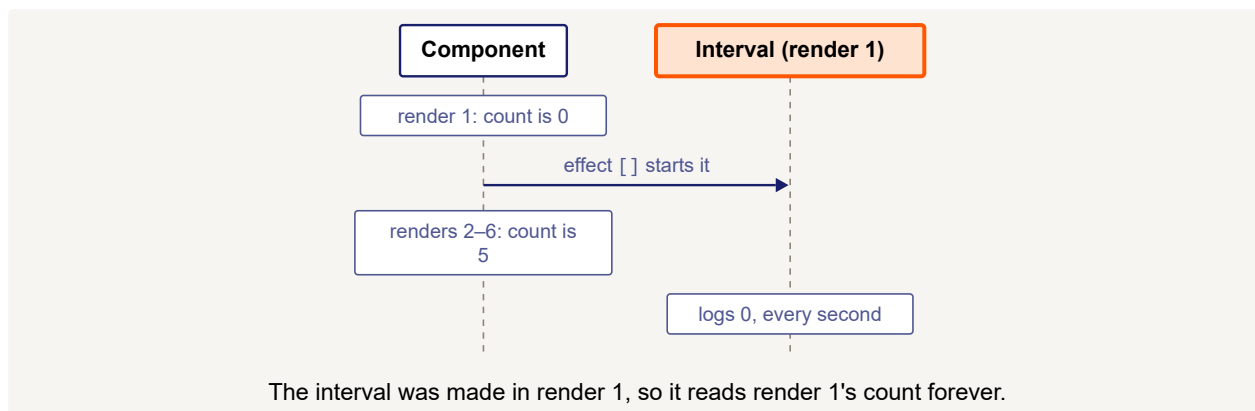
```

Example. The Try it box has three effects, one of each kind, and logs when each runs. Click **+1** and type in the input: after the click A and C run; after typing only A runs. The render-cycle visualiser has the same component as the scenario "No array, [] and [count]".

Common mistake. Removing a dependency to stop an effect from running "too often", for example deleting query from [query]. The effect stops re-running, but it now fetches the old query forever. If an effect runs too often, fix the reason (an object created in render, a value that should be derived), never the array.

Stale closures in effects and timers

A function created during a render **closes over** that render's values; if an effect keeps it alive (an interval, a listener) without re-running, it keeps reading old values: a **stale closure**.



Fix 1: list the value [count]: every change cleans up the old interval and starts a new one with a fresh closure.

Fix 2: stop reading it If the callback only changes the state, give the setter an updater, `setCount((c) => c + 1)`. React passes the current value, so [] is truthful and one interval is enough.

Rare cases A callback that must read the latest value without restarting can read it from a ref (see [useRef](#)).

```

import { useState, useEffect } from 'react';

function StaleTicker({ count }) {
  useEffect(() => {
    const id = setInterval(() => console.log(`stale ticker sees ${count}`), 1000);
    return () => clearInterval(id);
  }, []); // count is missing: the linter would warn
  return <p>Stale ticker running</p>;
}

function FixedTicker({ count }) {
  useEffect(() => {
    const id = setInterval(() => console.log(`fixed ticker sees ${count}`), 1000);
    return () => clearInterval(id);
  }, [count]); // a new interval for every count
  return <p>Fixed ticker running</p>;
}

function App() {
  const [count, setCount] = useState(0);
  const [fixed, setFixed] = useState(false);
  return (
    <>
      <button onClick={() => setCount(count + 1)}>Count: {count}</button>
    </>
  );
}

```

```

    <label>
      <input type="checkbox" checked={fixed} onChange={(e) => setFixed(e.target.checked)} />
      Use the fixed ticker
    </label>
    {fixed ? <FixedTicker count={count} /> : <StaleTicker count={count} />}
  </>
);
}

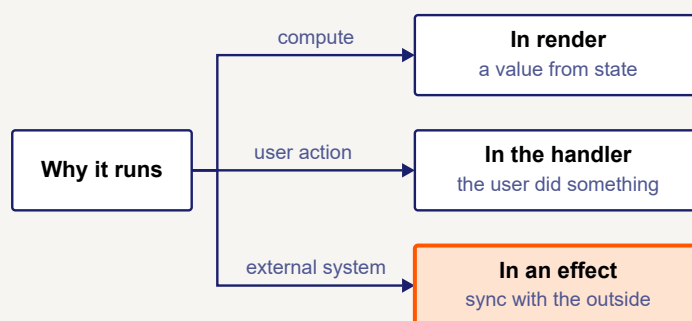
```

Example. In the Try it box, the button belongs to App, which passes count to a ticker. The **stale** ticker's effect has [], so its interval logs 0 forever while you click. Switch to the **fixed** ticker: its effect lists count, re-runs after every click, and the log follows the button.

Common mistake. Fixing a stale closure by moving the variable out of React (let latest = 0 at module level) or by reading it from the DOM. The component then has two sources of truth. Use the dependency array or an updater function first.

You might not need an effect

Effects are for synchronising with **external systems**: compute values from props or state during render, and respond to what the user did in the **event handler**.



Why the code runs decides where it goes.

Effect or not?

You want to...	Put it in...
Show tasks filtered by the search text	Render (a derived const)
Send a POST when the user clicks Save	The click or submit handler
Clear the form after the POST succeeds	The same handler, after await
Reset all state when an id prop changes	key={id} on the component (see State belongs to a place in the tree)
Tell the parent about a change	Call its callback in the handler that sets the state
Load the tasks of the user on screen	An effect with [userId] (or a data library)
Run a clock while the component is visible	An effect with a cleanup

```

// Not needed: an effect that reacts to "submitted"
useEffect(() => {
  if (submitted) { postTask(title); setTitle(''); }
}, [submitted]);

// Better: do it where it happens
async function handleSubmit(e) {
  e.preventDefault();
  await postTask(title);
  setTitle('');
}

```

```

import { useState } from 'react';

function App() {
  const [title, setTitle] = useState('');

```

```
const [created, setCreated] = useState([]);

async function handleSubmit(e) {
  e.preventDefault();
  const res = await fetch('/api/tasks', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title, done: false, userId: 1 }),
  });
  if (!res.ok) { console.log('could not create:', res.status); return; }
  const task = await res.json();
  setCreated((prev) => [...prev, task]); // respond to the event here...
  setTitle(''); // ...and reset here, no effect
}

return (
  <form onSubmit={handleSubmit}>
    <input value={title} onChange={(e) => setTitle(e.target.value)} placeholder="New task" />
    <button disabled={!title.trim()}>Add</button>
    <p>{created.length} {created.length === 1 ? 'task' : 'tasks'} created</p>
    <ul>{created.map((t) => <li key={t.id}>#{t.id} {t.title}</li>)}</ul>
  </form>
);
}
```

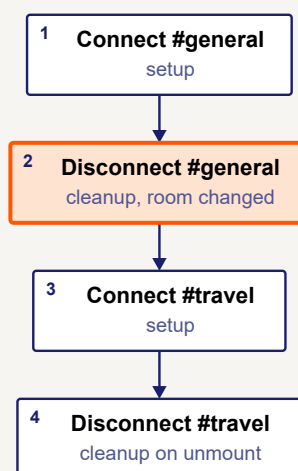
Example. The Try it box creates tasks against the mock API: the POST and the reset live in the submit handler, and the list title ("2 tasks") is derived. There is no effect in the component at all.

Common mistake. Using an effect to "watch" props and copy them into state: `useEffect(() => setName(user.name), [user])`. The first render shows the old name, a second render fixes it, and any edit the user made is overwritten when user changes. Read `user.name` directly, or reset the form with a key.

Cleanup, refs and StrictMode

Cleanup: before the next effect and on unmount

The function an effect returns is its **cleanup**: React runs it before the effect runs again (with the old values) and when the component is removed from the screen (unmount), to undo the setup.



The old cleanup always runs before the new setup, and once more on unmount.

Every setup that keeps running has a matching cleanup

Setup	Cleanup
<code>setInterval</code> , <code>setTimeout</code>	<code>clearInterval</code> , <code>clearTimeout</code>
<code>addEventListener</code> (type, fn)	<code>removeEventListener</code> (type, fn), with the same function
<code>connect</code> (), <code>subscribe</code> ()	<code>disconnect</code> (), <code>unsubscribe</code> ()

Setup	Cleanup
fetch	An ignore flag or <code>controller.abort()</code> (see Cancelling with AbortController)

```
import { useState, useEffect } from 'react';

function ChatRoom({ roomId }) {
  useEffect(() => {
    console.log(`connect to #${roomId}`);
    return () => console.log(`disconnect from #${roomId}`);
  }, [roomId]);
  return <h2>Welcome to #${roomId}</h2>;
}

function App() {
  const [roomId, setRoomId] = useState('general');
  const [show, setShow] = useState(true);
  return (
    <>
      <select value={roomId} onChange={(e) => setRoomId(e.target.value)}>
        <option value="general">general</option>
        <option value="travel">travel</option>
        <option value="music">music</option>
      </select>
      <button onClick={() => setShow(!show)}>{show ? 'Close the chat' : 'Open the chat'}</button>
      {show && <ChatRoom roomId={roomId} />}
    </>
  );
}
```

Example. In the Try it box, the chat room "connects" in an effect that depends on `roomId`. Change the room: the console shows the disconnect from the old room **before** the connect to the new one. Then click **Close the chat**: the last cleanup runs.

Common mistake. Writing the cleanup with a new function: `window.addEventListener('resize', () => update())` and later `window.removeEventListener('resize', () => update())`. Two arrow functions are two different objects, so nothing is removed. Keep the handler in a variable inside the effect and pass that same variable to both calls.

useRef: a box that survives renders without causing one

`useRef(initial)` returns an object { `current` } that React keeps for the component's whole life; you can read and change `.current` freely, and changing it does **not** re-render.

An element of the page `const inputRef = useRef(null)` and `<input ref={inputRef} />`: after the commit, React puts the DOM element in `inputRef.current`, so a handler can call `inputRef.current.focus()`.

A timer id Keep the id of `setInterval` in a ref, so a Stop button can clear it later.

The latest value An interval or listener that must not restart can read `latestRef.current`, updated in an effect or a handler, instead of a stale variable (see [Stale closures](#)).

State or ref?

	<code>useState</code>	<code>useRef</code>
Changing it	Re-renders	Does not re-render
Read during render	Yes: the snapshot of this render	Avoid: the screen would not follow it
Use it for	What the screen shows	DOM elements, timer ids, values only your code reads

```
import { useState, useRef } from 'react';

function App() {
  const [seconds, setSeconds] = useState(0);
  const intervalRef = useRef(null); // the timer id: never shown
```

```

const inputRef = useRef(null);      // the <input> element, after the commit
const clicksRef = useRef(0);        // changes without re-rendering
console.log('render');

function start() {
  if (intervalRef.current) return; // already running
  intervalRef.current = setInterval(() => setSeconds((s) => s + 1), 1000);
}
function stop() {
  clearInterval(intervalRef.current);
  intervalRef.current = null;
}
function countSilently() {
  clicksRef.current += 1;
  console.log('clicks so far:', clicksRef.current);
}

return (
  <>
    <p>{seconds} s</p>
    <button onClick={start}>Start</button> <button onClick={stop}>Stop</button>
    <p>
      <input ref={inputRef} placeholder="Name" />{' '}
      <button onClick={() => inputRef.current.focus()}>Focus the input</button>
    </p>
    <button onClick={countSilently}>Count without rendering</button>
  </>
);
}

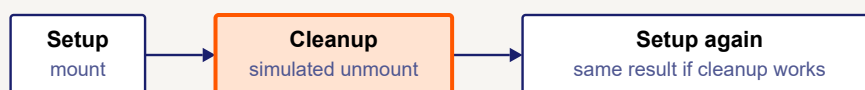
```

Example. In the Try it box, **Start** begins a timer whose id lives in a ref, and **Stop** clears it. **Focus the input** uses a DOM ref. **Count without rendering** increases `clicksRef.current`: the console logs the growing count, but no new "render" line appears, because changing a ref never re-renders.

Common mistake. Showing a ref on screen: `<p>{clicksRef.current} clicks</p>`. The number only changes when something else happens to re-render, because changing a ref never does. If the screen shows it, it is state.

StrictMode: why effects run twice in development

`<StrictMode>` is a development-only checker: on mount it runs every effect, its cleanup and the effect again (setup → cleanup → setup), and it renders each component twice; production builds do neither.



In development, every mount is tested: setup, cleanup, setup.

- **The double render** checks render code the same way: rendering must be pure, so two calls must give the same JSX. A counter incremented in the body or an array pushed during render shows up early. Since React 19 the console shows the logs of both renders.
- **Fix, never suppress:** add or correct the cleanup. Removing `<StrictMode>`, or guarding the effect with a "has it run already?" ref, hides the bug until production.
- **A fetch** is aborted or ignored by its cleanup, so the result is the same; the duplicate request only happens in development.

```

// main.jsx, as created by the Vite React template
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);

```

```

    </StrictMode>
  );

```

```

import { StrictMode, useEffect } from 'react';
import { createRoot } from 'react-dom/client';

function App() {
  console.log('render');
  useEffect(() => {
    console.log('connect');
    return () => console.log('disconnect');
  }, []);
  return <p>Connected (look at the console)</p>;
}

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);

```

Example. The Try it box renders its own root wrapped in `<StrictMode>` (that is why it calls `createRoot` itself). Its console shows `render` twice, then `connect`, `disconnect`, `connect`. Remove the `<StrictMode>` tags and each line appears once.

Common mistake. Adding a `didRun` flag to make an effect run once "because it runs twice". The effect would also run once without a cleanup, so the real bug (no cleanup) survives, and the flag breaks the remount that `StrictMode` is simulating.

Effects in practice

Fetching data in an effect

To load data for what is on the screen, start the request in an effect whose dependencies are the inputs of the request (`[]`, `[id]`, `[query]`), check `res.ok`, store the result in state, and let the cleanup discard an outdated answer.

No async effect function

An async function returns a promise, but an effect must return nothing or a cleanup. Define an async function inside the effect and call it.

`res.ok`

`fetch` rejects only when there is no response at all; a 404 or 500 still resolves. Check `res.ok` and throw, so one catch handles both kinds of failure (see [async and await](#)).

An ignore flag in the cleanup

The component may disappear, or a newer request may start, before the answer arrives. The cleanup sets `ignore = true`, and every `setState` checks it first.

```

useEffect(() => {
  let ignore = false;
  async function load() {
    try {
      const res = await fetch(`/api/tasks?userId=${userId}`);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      const data = await res.json();
      if (!ignore) setTasks(data);
    } catch (err) {
      if (!ignore) setError(err.message);
    }
  }
  load();
  return () => { ignore = true; }; // a newer run or an unmount wins
}, [userId]);

```

```
import { useState, useEffect } from 'react';

function App() {
  const [userId, setUserId] = useState(1);
  const [tasks, setTasks] = useState([]);
  const [error, setError] = useState(null);

  useEffect(() => {
    let ignore = false;
    async function load() {
      try {
        const res = await fetch(`/api/tasks?userId=${userId}`);
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const data = await res.json();
        if (!ignore) { setTasks(data); setError(null); }
      } catch (err) {
        if (!ignore) setError(err.message);
      }
    }
    load();
    return () => { ignore = true; };
  }, [userId]);

  return (
    <>
      <select value={userId} onChange={(e) => setUserId(Number(e.target.value))}>
        <option value={1}>User 1</option>
        <option value={2}>User 2</option>
        <option value={3}>User 3</option>
      </select>
      {error && <p role="alert">Could not load: {error}</p>}
      <ul>{tasks.map((t) => <li key={t.id}>{t.title}</li>)}</ul>
    </>
  );
}
```

Example. The Try it box loads the tasks of a user from the mock API (it answers `/api/...` after 300 ms and logs each request). Change the user: the effect re-runs with the new id, the cleanup of the previous run flips its `ignore` flag, and the list switches over.

Common mistake. Fetching in the component body: `fetch(url).then((r) => r.json()).then(setTasks)`. Every render starts a request, every response sets state, every state change renders again: an endless loop of requests. Fetches that load data for the screen go in an effect; fetches caused by a click go in the handler.

Custom hooks: reusing stateful logic

A custom hook is a function whose name starts with `use` and that calls other hooks: it packages stateful logic (state and effects) so several components can reuse it, each with its own independent state.

- **Logic, not state:** calling `useToggle()` in two components is like calling `useState` in both: two separate values. To share one value, lift it to a common parent or use context (see [Lifting state up](#)).
- **The `use` prefix** tells React's linter and readers that the function calls hooks, so the [rules of hooks](#) apply to it. A function that calls no hooks is a plain helper and should not start with `use`.
- **Any return shape:** a pair like `useState([on, toggle])` or an object for many values, such as the [data hook](#) `{ data, loading, error, reload }`.

```
function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = () => setOn((v) => !v);
  return [on, toggle];
}
```

```
// in any component
const [open, toggleOpen] = useToggle();
```

```
import { useState } from 'react';

function useToggle(initial = false) {
  const [on, setOn] = useState(initial);
  const toggle = () => setOn(v => !v);
  return [on, toggle];
}

function Panel({ title, children }) {
  const [open, toggleOpen] = useToggle();
  return (
    <section>
      <button aria-expanded={open} onClick={toggleOpen}>{open ? '▼' : '▶'} {title}</button>
      {open && <p>{children}</p>}
    </section>
  );
}

function App() {
  return (
    <>
      <Panel title="What is state?">Data a component remembers between renders.</Panel>
      <Panel title="What is an effect?">Code that synchronises with something outside React.</Panel>
    </>
  );
}
```

Example. The Try it box defines `useToggle` and uses it in two `Panel` components. Opening one does not open the other: each call has its own state.

Common mistake. Expecting two components that call the same custom hook to see the same data, for example a `useCart()` in the header and in the product page. Each call has its own state, so adding to one cart does not change the other. Shared data needs one owner: a common parent or a context provider.

A custom hook for persistence: `useLocalStorage`

`useLocalStorage(key, initial)` **works like** `useState`, **but reads its first value from** `localStorage` **and writes every change back, so the value survives a page reload.**

- **Strings only:** save with `JSON.stringify(value)` and read with `JSON.parse(text)` (see [JSON](#)).
- **Wrap both in try/catch:** storage can be full, disabled (private browsing, strict privacy settings, sandboxed frames) or hold text that is not valid JSON. Fall back to the initial value instead of crashing.
- **Typical uses:** a theme or language preference, a draft that should survive a reload, the last filters of a list. Keeping a user logged in after a reload builds on the same idea (see [Staying logged in after a reload](#)).
- **Nothing secret:** any script on the page can read it.

```
function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    try {
      const saved = localStorage.getItem(key);
      return saved === null ? initial : JSON.parse(saved);
    } catch {
      return initial; // blocked or invalid: use the default
    }
  });

  useEffect(() => {
    try { localStorage.setItem(key, JSON.stringify(value)); } catch { /* full or blocked */ }
  }, [key, value]);
}
```

```

    return [value, setValue];
  }

// usage: exactly like useState
const [theme, setTheme] = useLocalStorage('theme', 'light');

```

```

import { useState, useEffect } from 'react';

function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    try {
      const saved = localStorage.getItem(key);
      return saved === null ? initial : JSON.parse(saved);
    } catch {
      console.log(`storage blocked: "${key}" starts from its default`);
      return initial;
    }
  });

  useEffect(() => {
    try { localStorage.setItem(key, JSON.stringify(value)); }
    catch { console.log(`storage blocked: "${key}" kept in memory only`); }
  }, [key, value]);

  return [value, setValue];
}

function App() {
  const [theme, setTheme] = useLocalStorage('theme', 'light');
  const [draft, setDraft] = useLocalStorage('draft', '');
  return (
    <div style={{ padding: 8, background: theme === 'dark' ? '#222' : '#fff', color: theme === 'dark' ? '#eee' :
    '#111' }}>
      <button onClick={() => setTheme(theme === 'dark' ? 'light' : 'dark')}>Theme: {theme}</button>
      <p><textarea value={draft} onChange={(e) => setDraft(e.target.value)} placeholder="Draft note" /></p>
    </div>
  );
}

```

Example. The Try it box uses the hook for a theme choice and a draft note. The preview runs in a locked-down frame where storage is blocked, so the catch branches run (the console says so) and the values reset when the preview reloads; in your own app they survive a page reload. That is exactly why the try/catch is there.

Common mistake. Reading storage in the body without the lazy initialiser:

`useState(JSON.parse(localStorage.getItem('tasks')))`. It parses on every render (the result is ignored after the first), and the first time it runs `JSON.parse(null)`, which returns `null` instead of your default, so `tasks.map` crashes.